Building REST APIs with FastAPI

Secure, Scalable, and Documented

In this module, we'll explore how to build powerful REST APIs with FastAPI, covering everything from core REST principles and OpenAPI documentation to CRUD operations with data validation and JWT authentication implementation.

By the end of this session, you'll be able to create professional-grade APIs that are both secure and well-documented.

What is REST?

REST Definition

REpresentational **S**tate **T**ransfer - an architectural style for designing networked applications.

Core Principles

Stateless communication - server doesn't store client state

Resource-based URLs - clear paths like `/books`, `/users`

Standard HTTP methods for operations

GET

Retrieve resources

POST

Create resources

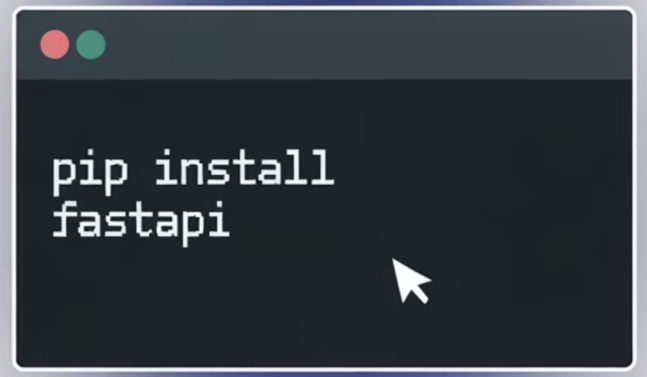
PUT/PATCH

Update resources

DELETE

Remove resources

Activity: Identify REST actions in the BookHub API example



FastAPI Setup

Step-by-Step Installation

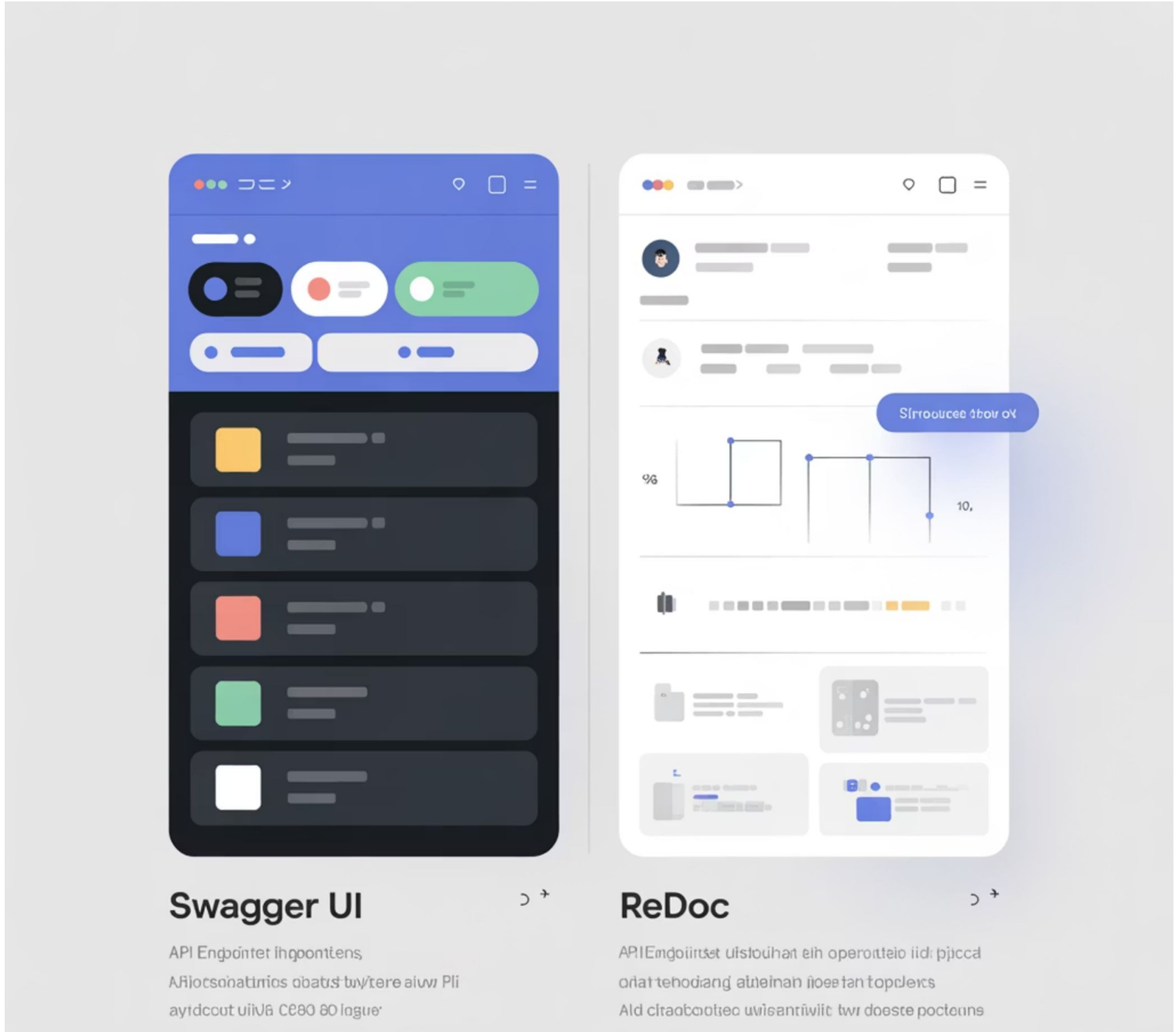
```
# Create virtual environmentpython -m venv venv# Activate (Windows)venv\Scripts\activate# Install packagespip install fastapi uvicorn
```

Your First Endpoint

```
from fastapi import FastAPIapp = FastAPI()@app.get("/")def home():    return {"message": "Welcome!"}# Run with:# uvicorn main:app --reload
```

Instructor Note: This will be demonstrated live in VS Code, so follow along and be ready to troubleshoot installation issues.

Interactive API Documentation



Auto-Generated Docs

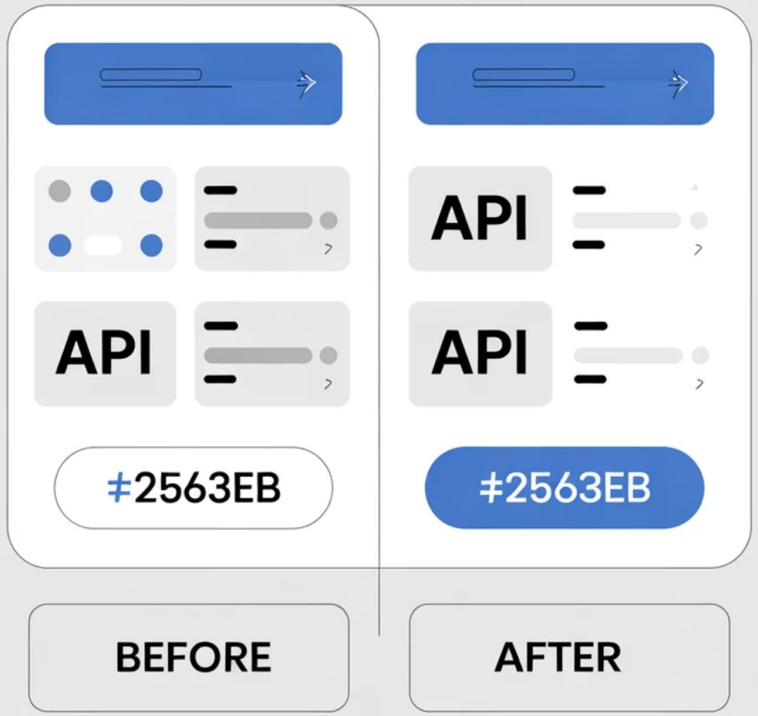
`/docs` - Swagger UI

`/redoc` - Alternative docs

Key Features

- Test endpoints directly
- See request/response schemas
- Authentication testing
- Zero extra coding required

Data Validation Magic



Pydantic Models

```
from pydantic import BaseModel, Fieldclass Book(BaseModel):    id: int    title:
str = Field(min_length=3)    genre: str = "General"
```

Key Benefits

- Automatic error messages
- Data type enforcement
- Default values
- Schema generation for docs

CRUD Operations

CREATE

```
@app.post("/books")def add_book(book: Book):    books_db.append(book)    return book
```

READ

```
@app.get("/books/{id}")def get_book(id: int):    for book in books_db:        if book.id
== id:            return book    raise HTTPException(        status_code=404,
detail="Book not found"    )
```

UPDATE

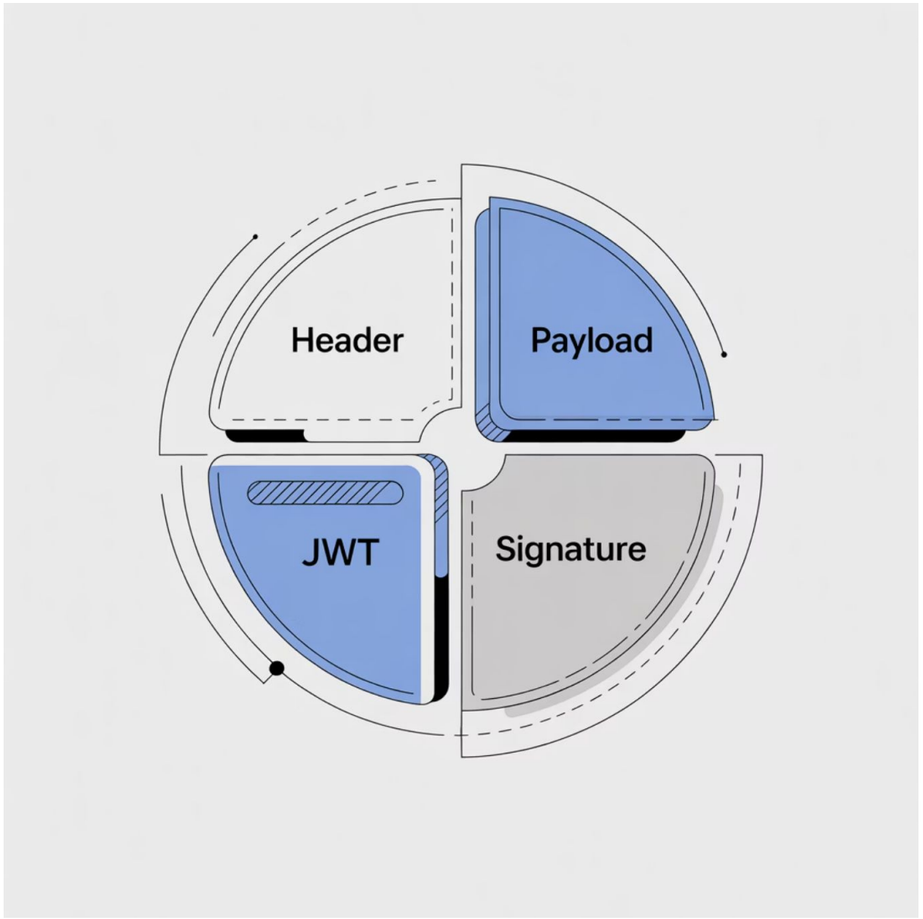
```
@app.put("/books/{id}")def update_book(id: int, book: Book):    # Find and update book
return updated_book
```

DELETE

```
@app.delete("/books/{id}")def delete_book(id: int):    # Find and remove book    return
{"message": "Deleted"}
```

Activity: Implement the GET by ID endpoint in your local environment.

Authentication Fundamentals



Why JWT?

- Stateless authentication
- Digital signature verification
- Self-contained payload with user claims
- Cross-domain compatibility

Login

User submits credentials

JWT Creation

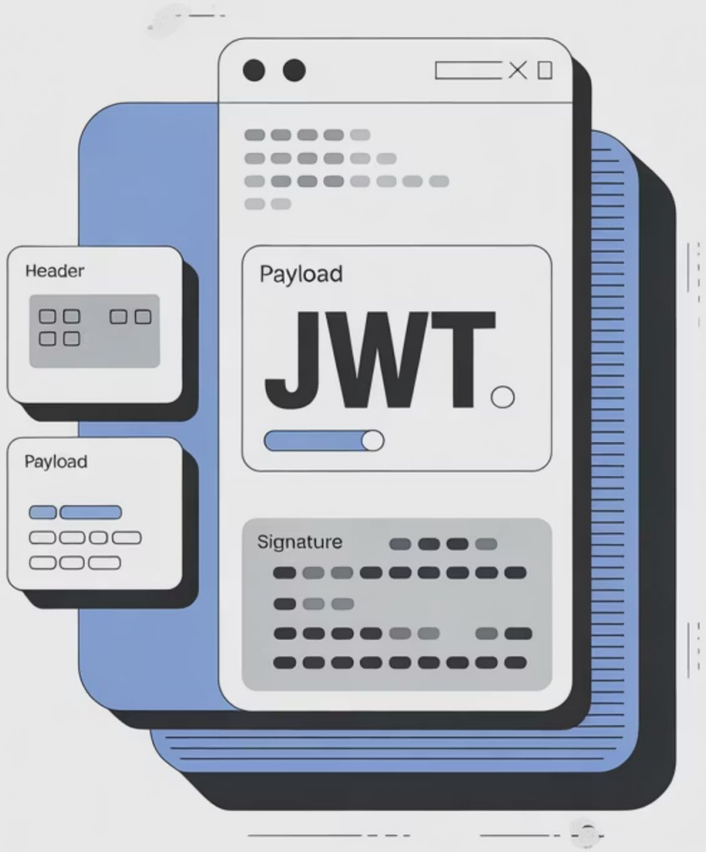
Server validates & creates signed token

Password Security

Never Store Plain Text!

```
from passlib.context import CryptContextpwd_context = CryptContext(    schemes=["bcrypt"])#
Hashinghashed = pwd_context.hash("mypassword")# $2b$12$EixZaYVK1fsbw1ZfbX3OXe...#
Verificationis_valid = pwd_context.verify(    "attempted_password",     hashed_password) # Returns
True/False
```

Critical Security Point: Even with hashing, you should never log passwords or store them in insecure locations. Bcrypt automatically handles salting to prevent rainbow table attacks.



Token Implementation

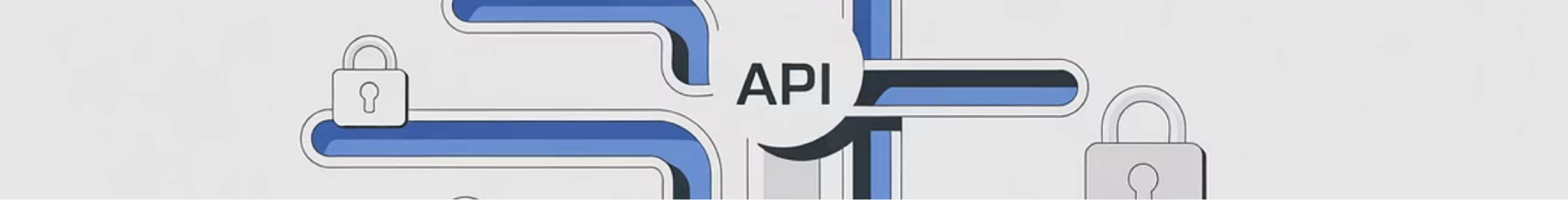
Token Generation

```
from jose import jwtfrom datetime
import datetime, timedeltatoken =
jwt.encode(    {        "sub":
"user123",  # subject (user id)
"exp": datetime.utcnow() +
timedelta(minutes=30)    },
"SECRET_KEY",    algorithm="HS256")
```

Token Verification

```
try:    payload = jwt.decode(
token,         "SECRET_KEY",
algorithms=["HS256"]    )
user_id = payload.get("sub")except
JWTError:    raise HTTPException(
status_code=401,
detail="Invalid token"    )
```

Activity: Decode a sample token at jwt.io to see the internal structure.



Protecting Endpoints

1. Create OAuth2 Scheme

```
from fastapi.security import
OAuth2PasswordBeareroauth2_scheme =
OAuth2PasswordBearer(    tokenUrl="login")
```

2. Implement User Extraction

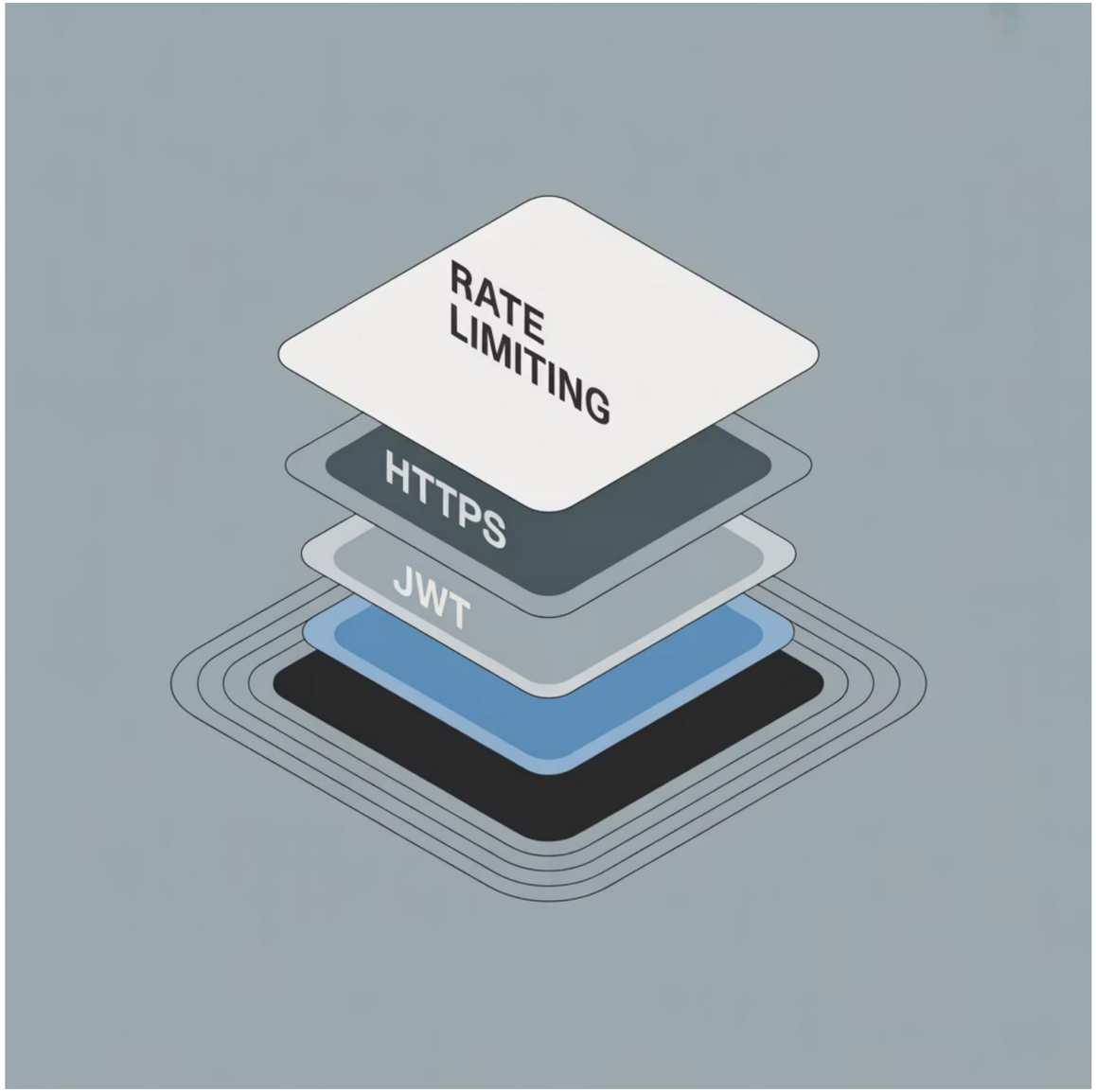
```
def get_current_user(    token: str =
Depends(oauth2_scheme)):    # Verify token
payload = jwt.decode(token, SECRET_KEY,
algorithms=[ALGORITHM])    return payload
```

3. Protect Routes with Dependency

```
@app.delete("/books/{id}")def delete_book(
id: int,    user: dict =
Depends(get_current_user)):    if
user["role"] != "admin":        raise
HTTPException(status_code=403)    # Delete
book logic
```

Instructor Note: We'll demonstrate Thunder Client authentication setup for testing protected endpoints.

Real-World Security



Best Practices

- Use HTTPS in production
- Store secrets in environment variables
- Set short token expiration (15-30 mins)
- Implement refresh tokens
- Add rate limiting

Common Pitfalls

- Hardcoded secrets in source code
- Long token TTL (time to live)
- Missing input validation
- Inadequate error handling

Q&A and Recap

Key Takeaways

REST uses HTTP effectively but isn't limited to it

Pydantic validation beats manual validation every time

JWT enables stateless authentication

FastAPI docs accelerate development and testing

Quiz

What status code for invalid credentials? (401)

How to add description to Pydantic field? (Field(..., description="..."))

What's the risk of long JWT expiration? (Prolonged access if token is compromised)

Next Activity

Build a complete user registration endpoint with password hashing and validation.